

Full-Stack & DevOps Engineer Take-Home Assessment

Overview

You'll be building a **Dynamic Form Builder** application with a React frontend, Laravel backend API, and containerized deployment setup. This assessment evaluates your full-stack development capabilities, from pixel-perfect UI to API design and DevOps practices.

The Challenge

Build a complete form builder application that allows users to:

- Create custom forms through a drag-and-drop interface (Frontend)
- Authenticate and manage their forms via API (Backend)
- Deploy the application using Docker and CI/CD (DevOps)

Figma Link: <https://www.figma.com/design/XmIFCa1X4h5kq2iTBPpYkB/FE-Engineer-T1>

Password: Abujaloga

Part 1: Frontend Development (React)

Expected User Workflow

The application should support the following intuitive user flow:

1. **Authentication:** Users must log in to access the form builder
2. **Create Sections:** Users click to add new form sections with custom titles
3. **Add Groups:** Within each section, users can add multiple groups to organize related fields
4. **Drag & Drop Elements:** Users drag field types from the sidebar and drop them into specific groups
5. **Save Forms:** Forms are persisted to the backend via API

Core Requirements

Form Structure Management

- **Sections:** Users can add/remove form sections with titles
- **Groups:** Within sections, users can create titled groups of related fields
- **Fields/Elements:** Each group contains various form field types

Field Types to Implement

- Text input (Last Name, Email Address)
- Radio buttons (Yes/No options)
- Checkboxes (Multiple selection options)
- File upload (with drag-and-drop)
- Dropdown/Select

User Interface Features

- **Authentication UI:** Login and registration forms
- **Drag and Drop:** Reorder sections, groups, and fields
- **Element Sidebar:** Right panel showing available field types
- **Pixel-Perfect Design:** Match the Figma design - spacing, colors, typography, shadows, borders
- **Loading States:** Show appropriate feedback during API calls
- **Error Handling:** Display user-friendly error messages

State Management

- Maintain form structure in application state
- Handle nested data (sections → groups → fields)
- Implement undo/redo functionality
- Sync with backend API

Technical Requirements

- React 18+ with hooks and functional components
- SASS/SCSS for styling
- Ant Design components where applicable
- Redux Toolkit for state management
- React DnD or @dnd-kit for drag-and-drop
- Axios for API calls

Part 2: Backend Development (Laravel)

API Requirements

Build a RESTful API using Laravel that supports:

Authentication Endpoints

POST /api/register - User registration
POST /api/login - User login (returns JWT token)
POST /api/logout - User logout
GET /api/user - Get authenticated user details

Form Management Endpoints

GET /api/forms - List all forms for authenticated user
POST /api/forms - Create a new form
GET /api/forms/{id} - Get specific form details
PUT /api/forms/{id} - Update form structure
DELETE /api/forms/{id} - Delete a form

Backend Requirements

Authentication

- Implement JWT authentication using [tymon/jwt-auth](#) or Laravel Sanctum
- Protect form endpoints with authentication middleware
- Handle token refresh appropriately

Database Schema

Design appropriate database tables for:

- **users** - Standard user authentication
- **forms** - Form metadata (title, description, user_id, timestamps)
- **form_structure** - JSON column storing the complete form structure (sections, groups, fields)

Validation

- Validate all incoming requests
- Return appropriate error messages
- Handle nested JSON structure validation for form data

API Response Format

Maintain consistent JSON response format:

```
{
  "success": true,
  "data": { },
  "message": "Operation successful"
}
```

Technical Requirements

- Laravel 10+
 - MySQL or PostgreSQL database
 - RESTful API design principles
 - Proper error handling and logging
 - API documentation (simple README is acceptable)
-

Part 3: DevOps & Deployment

Docker Setup

Create a containerized environment with:

Required Services

1. **Frontend Container** (React app served via Nginx)
2. **Backend Container** (Laravel API with PHP-FPM)
3. **Database Container** (MySQL/PostgreSQL)
4. **Nginx Container** (Reverse proxy)

Docker Requirements

- Create a `docker-compose.yml` file for local development
- Separate `Dockerfile` for frontend and backend
- Configure proper networking between containers
- Use environment variables for configuration
- Include volume mounts for development

CI/CD Pipeline

Implement a basic CI/CD pipeline using **GitHub Actions**:

Pipeline Requirements

.github/workflows/deploy.yml

Stages:

1. Build
 - Install dependencies
 - Run frontend build
 - Run Laravel optimizations
2. Test (Optional but recommended)
 - Run frontend tests (if implemented)
 - Run backend tests (if implemented)

3. Docker Build
 - Build Docker images
 - Tag images appropriately
4. Deploy (to staging/demo)
 - Push images to Docker registry
 - Deploy to cloud instance (or document the process)

Deployment Documentation

Provide clear documentation for:

- How to run the application locally with Docker
- Environment variables required
- Database migration steps
- Deployment process
- Any assumptions or limitations

Live Deployment (Required)

You must deploy your application and provide a live URL in your PR submission.

Free Hosting Options

Choose any of these platforms to deploy your application:

For Full-Stack Apps (Frontend + Backend + Database):

- **Railway.app** (Recommended)
 - Free tier: 500 hours/month, \$5 credit
 - Supports Docker, databases included
 - Easy GitHub integration
 - railway.app
- **Render.com**
 - Free tier for web services and PostgreSQL
 - Auto-deploy from GitHub
 - Docker support
 - render.com
- **Fly.io**
 - Free tier: 3 shared VMs, 3GB storage
 - Excellent Docker support
 - Global deployment
 - fly.io

Alternative (Frontend + Backend Separate):

- **Frontend:** Vercel, Netlify, or Cloudflare Pages (all free)
- **Backend:** Railway, Render, or Fly.io (free tiers)
- **Database:** Railway, Render, or Supabase (free tiers)

Deployment Checklist

Your deployed application must have:

- ☒ Working authentication (register & login)
- ☒ Ability to create and save at least one form
- ☒ Persistent database (data survives container restarts)
- ☒ HTTPS enabled (most free platforms provide this)
- ☒ Environment variables properly configured

Deployment Documentation Required

In your README.md, include:

- Live application URL
- Test credentials (if you want to provide demo access)
- Deployment platform used
- Any deployment-specific configurations
- Known issues in production (if any)

Simplified Scope (Keep It Simple)

To keep this assessment manageable within a reasonable timeframe:

Must Have

- ☒ User authentication (login/register)
- ☒ Create and save at least one form
- ☒ Retrieve saved forms
- ☒ Basic form structure (sections → groups → fields)
- ☒ Drag and drop for at least 2 field types
- ☒ Docker setup with docker-compose
- ☒ Basic GitHub Actions workflow

Nice to Have (Optional)

- Form deletion and updates
- Undo/redo functionality
- Complete pixel-perfect design
- Comprehensive test coverage
- Advanced CI/CD with automated deployments

Submission Requirements

How to Submit

1. **Fork the repository:** <https://github.com/paxpass/fullstack-devops-assessment>
2. **Create a feature branch:** `feature/fullstack-form-builder-[your-name]`

Project Structure:

```
/frontend      # React application
/backend       # Laravel application
/docker        # Docker configuration files
.github/workflows # CI/CD pipeline
README.md      # Main documentation
```

3. **Submit a Pull Request** to the `main` branch
4. **PR Title:** "Full-Stack Assessment - [Your Name]"
5. **PR Description Must Include:**
 - **Live deployment URL** (Required)
 - Brief implementation overview
 - Key technical decisions made
 - Trade-offs and simplifications made
 - Instructions for running locally with Docker
 - Instructions for testing the application
 - API documentation or Postman collection
 - Any assumptions or clarifications
 - Time spent on the assessment
 - Deployment platform used and why

Documentation Requirements

Include in your README.md:

```
# Form Builder Application
```

```
## Live Demo
```

```
**URL**: [Your deployed application URL]
```

```
**Test Credentials** (optional):
```

```
- Email: demo@example.com
```

```
- Password: demo123
```

Quick Start with Docker
[Step-by-step instructions]

API Documentation
[Endpoint list with examples]

Frontend Features
[What you implemented]

Technical Decisions
[Your approach and reasoning]

Deployment
****Platform****: [Railway/Render/Fly.io/etc.]
****Why this platform****: [Brief explanation]
****Deployment steps****: [How you deployed]

Environment Variables
[Required configuration]

Known Limitations
[What's not included and why]

Good luck! We're excited to see your solution!